Ứng dụng thuật toán Random Forest phân tán dự đoán nguyên nhân trễ chuyến bay thương mại tại Hoa Kỳ năm 2024

Distributed Random Forest for Flight Delay Cause Prediction in US Commercial Aviation in 2024

GVHD: TS. Phan Hồ Viết Trường

Cù Văn Vĩ An
MSSV: 2001230005
Khoa Công nghệ Thông tin
Trường ĐH Công Thương TP.HCM
2001230005@huit.edu.vn

Lê Đức Lương
MSSV: 2001230490
Khoa Công nghệ Thông tin
Trường ĐH Công Thương TP.HCM
2001230490@huit.edu.vn

Trần Huỳnh Tuấn Anh
MSSV: 2001230023
Khoa Công nghệ Thông tin
Trường ĐH Công Thương TP.HCM
2001230023@huit.edu.vn

TÓM TẮT

Đồ án nghiên cứu ứng dụng Random Forest phân tán để phân loại trạng thái đúng giờ và nguyên nhân trễ chuyến bay thương mại tại Hoa Kỳ năm 2024. Dữ liệu gồm 7.079.081 bản ghi gốc và 6.965.267 bản ghi sau làm sạch. Nhóm xây dựng hai nhánh thực nghiệm: huấn luyện sáu mô hình trên CPU của một máy và triển khai các thuật toán Spark MLlib trên cụm máy kết nối qua Tailscale. Kết quả ngày 10/10/2026 cho thấy Random Forest CPU đạt Accuracy 79,18%, Weighted F1 69,98% và Macro F1 14,73%. Random Forest Spark đạt Accuracy 79,20%, Weighted F1 70,00%, với thời gian huấn luyện wall-clock 31,03 giây; bản CPU mất 85,18 giây. Đây là đối chiếu giữa hai cấu hình thực nghiệm, chưa phải phép đo tách riêng lợi ích của phân tán. CatBoost đạt Weighted F1 cao nhất trong nhóm CPU là 70,50%, còn LightGBM có Macro F1 cao nhất là 15,87%. Phân bố nhãn và ma trận nhầm lẫn cho thấy các mô hình vẫn nhận diện kém những nguyên nhân trễ ít gặp. Báo cáo trình bày cách tạo nhãn, đặc trưng, triển khai Random Forest, so sánh với baseline và phục vụ dự đoán qua Web_Air. Ba tích hợp boosting Spark trong bộ kết quả cũ không được sử dụng như số liệu huấn luyện thực đo.

Từ khóa

Random Forest; Apache Spark; dữ liệu lớn; chuyến bay; phân loại đa lớp; Parquet; Spark MLlib.

Phạm vi và đóng góp của đồ án

Nhóm tập trung vào việc xử lý dữ liệu lớn, triển khai Random Forest và đánh giá chất lượng dự đoán cùng thời gian chạy. Các baseline giúp xác định vị trí của thuật toán đề tài trong một bài toán cụ thể. Báo cáo không đề xuất thuật toán Random Forest mới và không xem việc có nhiều máy tự động bảo đảm mô hình chính xác hơn.

Phần phương pháp được đối chiếu với mã ETL, script huấn luyện và tài nguyên mô hình đã lưu. Phần kết quả sử dụng các file JSON và CSV của ngày 10/10. Những cập nhật chương trình sau đó được mô tả riêng như trạng thái triển khai hiện tại. Cách phân biệt này giúp tránh gắn số liệu của lần chạy trước cho mô hình hoặc cấu hình mới.

I GIỚI THIỆU VÀ MỤC TIÊU NGHIÊN CỨU

A Bài toán thực tế

Trễ chuyến bay ảnh hưởng đến lịch trình hành khách, khả năng khai thác tàu bay và điều phối hoạt động sân bay. Một chuyến bay có thể chậm vì hãng hàng không, thời tiết, hệ thống không phận, an ninh hoặc máy bay từ chặng trước đến muộn. Khi nhiều nguyên nhân cùng xuất hiện, nhãn của đồ án chọn nguyên nhân có số phút trễ lớn nhất. Đây là một quy tắc chuyển dữ liệu báo cáo thành mục tiêu học máy, không phải kết luận nhân quả về từng chuyến bay.

Khác với bài toán chỉ phân biệt trễ và đúng giờ, đồ án sử dụng sáu lớp. Lớp OnTime bao gồm chuyến đúng giờ hoặc có độ trễ đến dưới 15 phút. Các chuyến còn lại được gắn một trong năm nhóm nguyên nhân. Vì OnTime chiếm phần lớn dữ liệu, đánh giá một mô hình chỉ bằng tỷ lệ dự đoán đúng có thể che khuất hạn chế ở các lớp nhỏ. Báo cáo vì vậy kết hợp Accuracy, Weighted F1, Macro F1 và ma trận nhầm lẫn.

B Lý do chọn Random Forest

Random Forest kết hợp nhiều cây quyết định và phù hợp với dữ liệu dạng bảng có quan hệ phi tuyến [1]. Thuật toán không đòi hỏi mọi đặc trưng phải tác động tuyến tính đến xác suất đầu ra. Cấu trúc cây cũng cho phép khảo sát tầm quan trọng đặc trưng. Trong môi trường Spark, quá trình thống kê phục vụ xây cây có thể thực hiện trên các phân vùng dữ liệu, tạo cơ sở để nghiên cứu xử lý phân tán.

Tuy nhiên, ưu điểm lý thuyết không bảo đảm Random Forest luôn thắng các baseline. Độ sâu cây, số cây, cách mã hóa danh mục và phân bố nhãn đều ảnh hưởng đến kết quả. Đồ án chọn Random Forest làm trọng tâm để tìm hiểu cách triển khai và đánh giá; việc lựa chọn này không thay thế bằng chứng thực nghiệm. Khi baseline cho kết quả tốt hơn, nhóm trình bày trực tiếp thay vì điều chỉnh kết luận theo tên đề tài.

C Các câu hỏi thực nghiệm

Câu hỏi thứ nhất là Random Forest đạt chất lượng dự đoán và thời gian huấn luyện như thế nào so với Logistic Regression, Decision Tree và ba phương pháp boosting trên CPU. Câu hỏi thứ hai là các chỉ số thay đổi ra sao khi triển khai Random Forest bằng Spark MLlib. Câu hỏi thứ ba là Accuracy cao có đi kèm khả năng phân biệt các nguyên nhân trễ ít gặp hay không.

Các câu hỏi này được trả lời bằng dữ liệu đã lưu, với giới hạn của phép so sánh được nêu cùng kết quả. Hai nhánh chưa được chứng minh sử dụng đúng cùng từng bản ghi kiểm tra và cấu hình tương đương. Do đó, kết quả phản ánh hai thiết lập đang có, chưa đủ để tách riêng tác động của số máy, thư viện, siêu tham số và cách tổ chức dữ liệu.

D Cách tổ chức nghiên cứu

Nhóm bắt đầu từ kiểm tra dữ liệu, xử lý bản ghi thiếu thông tin và tạo nhãn. Tiếp theo là chuẩn bị đặc trưng, huấn luyện mô hình, đánh giá và lưu tài nguyên. Web_Air sử dụng mô hình CPU đã lưu để minh họa dự đoán, đồng thời đọc file kết quả để trình bày thực nghiệm. Những bước này tạo một chuỗi xử lý có thể kiểm tra từ đầu vào đến đầu ra.

Đóng góp chính nằm ở việc xử lý toàn bộ bộ dữ liệu hơn bảy triệu dòng và đối chiếu cách triển khai một thuật toán trên máy đơn với môi trường Spark. Việc đưa kết quả lên web giúp trình bày đồ án và thử dữ liệu đầu vào cụ thể. Độ trễ của ví dụ web được xem là phép đo minh họa tại thời điểm sử dụng, không thay thế benchmark trên tập kiểm tra.

E Vì sao cần đối chiếu cả chất lượng và thời gian

Một hệ thống dự đoán có thể dùng mô hình theo hai cách: đánh giá ngoại tuyến trên dữ liệu đã có và suy luận trực tuyến cho đầu vào của người dùng. Ở bước ngoại tuyến, thời gian fit ảnh hưởng đến khả năng thử cấu hình hoặc cập nhật model. Ở bước trực tuyến, chi phí dự đoán và khả năng nạp tài nguyên ảnh hưởng đến trải nghiệm sử dụng. Hai vấn đề này không nên gộp thành một chỉ số tốc độ duy nhất.

Trong đồ án, Spark được xem xét chủ yếu ở bước xử lý và huấn luyện dữ liệu lớn. Web dùng model CPU đã lưu để tránh khởi tạo một cụm tính toán cho từng yêu cầu đơn lẻ. Đây là lựa chọn triển khai của hệ thống hiện tại, không phải kết luận rằng mô hình Spark không thể phục vụ suy luận. Muốn mở thêm suy luận Spark trên Web cần lưu và nạp đúng pipeline đặc trưng, đồng thời kiểm soát tài nguyên SparkSession.

Quy mô dữ liệu và chất lượng mục tiêu cũng là hai khía cạnh khác nhau. Sáu triệu dòng sạch giúp khảo sát một khối lượng tính toán lớn, nhưng không tự bảo đảm các nhãn hiếm được học tốt. Ví dụ, nếu Security chỉ xuất hiện ở một tỷ lệ rất nhỏ thì tổng số mẫu lớn vẫn có thể tạo một bài toán khó về mất cân bằng. Vì vậy, tiêu chí dữ liệu lớn được chứng minh bằng số bản ghi, còn tiêu chí chất lượng được chứng minh bằng đánh giá từng lớp.

Việc so sánh baseline có vai trò kiểm tra kỳ vọng đối với thuật toán đề tài. Nếu một mô hình đơn giản cho chất lượng gần bằng Random Forest với thời gian thấp hơn, đây là kết quả cần phân tích. Báo cáo không mặc định kiến trúc phức tạp hoặc tên thuật toán là bằng chứng ưu thế. Kết luận cuối cùng chỉ được đưa ra trong phạm vi dữ liệu, đặc trưng và cấu hình thực nghiệm đã xác định.

II CƠ SỞ LÝ THUYẾT RANDOM FOREST

A Cây quyết định và độ hỗn tạp Gini

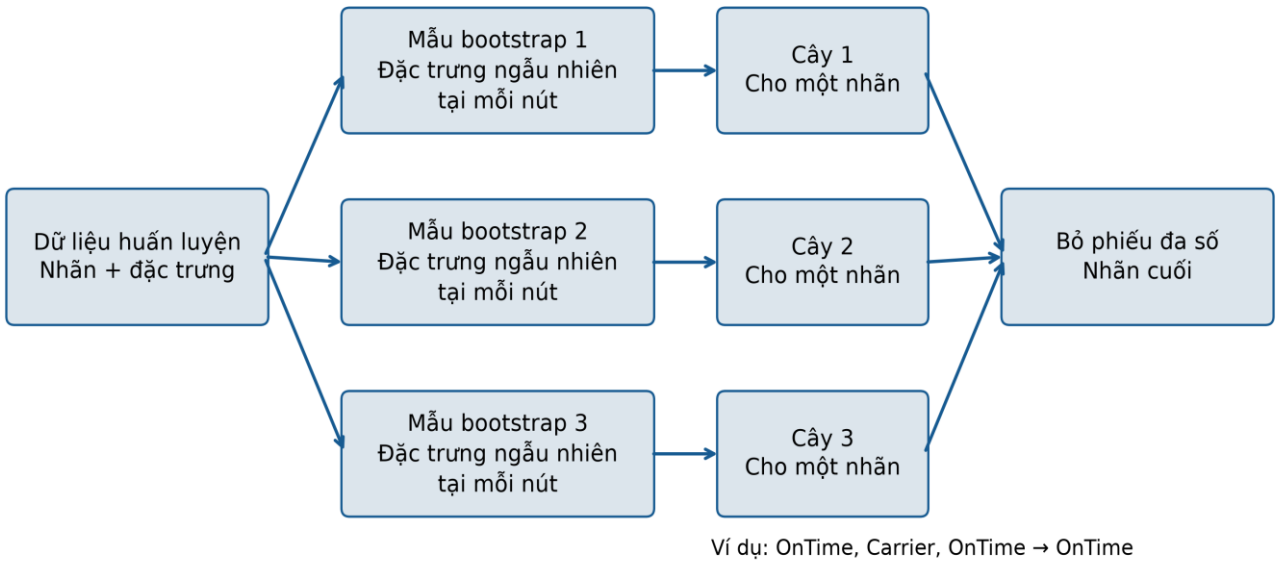
Một cây quyết định chia tập mẫu thành các nhánh dựa trên điều kiện của đặc trưng. Tại nút lá, cây đưa ra nhãn dự đoán hoặc phân bố xác suất. Ví dụ, cây có thể kiểm tra giờ khởi hành, tháng và hãng bay. Điều kiện cụ thể được học từ dữ liệu; sơ đồ nguyên lý không biểu diễn cây trích từ mô hình đã huấn luyện.

Độ hỗn tạp Gini đo mức pha trộn nhãn tại một nút. Với p_k là tỷ lệ lớp k trong tập S, Gini được xác định theo công thức $\text{Gini}(S) = 1 - \Sigma\, p_k^2$. Nếu mọi mẫu thuộc cùng một lớp thì Gini bằng không. Một điểm chia được đánh giá bằng mức giảm Gini có trọng số theo số mẫu ở hai nhánh. Thuật toán lựa chọn điểm chia phù hợp trong các ứng viên được xét.

B Tạo sự khác biệt giữa các cây

Random Forest không chỉ lặp lại một cây nhiều lần. Mỗi cây học từ dữ liệu được lấy mẫu và chỉ xét một tập con đặc trưng tại mỗi nút. Hai cơ chế này làm các cây khác nhau, giảm sự phụ thuộc của mô hình vào một cách chia duy nhất. Với phân loại, dự đoán cuối được tổng hợp từ các cây; cách hiện thực cụ thể có thể dựa trên phiếu hoặc xác suất lớp [1].

Nếu các cây có sai số không hoàn toàn giống nhau, tổng hợp dự đoán có thể ổn định hơn một cây đơn. Ngược lại, nếu dữ liệu không cung cấp đủ tín hiệu để nhận diện lớp trễ thì tăng số cây không tự tạo thêm thông tin. Rừng cũng có thể cùng nghiêng về lớp đông nhất. Vì vậy, cần đọc ma trận nhầm lẫn thay vì suy luận rằng một mô hình tổ hợp luôn tốt trên mọi lớp.



Hình 1. Nguyên lý Random Forest. Ví dụ bỏ phiếu chỉ minh họa cách tổng hợp dự đoán.

C Những tham số ảnh hưởng trực tiếp

Số cây tác động đến chi phí xây dựng và suy luận. Độ sâu giới hạn độ phức tạp của mỗi cây. Số đặc trưng xét tại một nút tác động đến sự đa dạng và khả năng tìm điểm chia. Trong mô hình CPU đã lưu, nhóm kiểm tra được 50 cây và độ sâu tối đa 10. Đây là cấu hình thực nghiệm của đồ án, không phải lựa chọn tối ưu đã được chứng minh qua tìm kiếm siêu tham số.

Hình 1 tách nguyên lý học và tổng hợp dự đoán khỏi cách bố trí máy tính. Một cây không đồng nghĩa với một Worker. Khi chuyển sang Spark, cách tổ chức tính toán thay đổi, còn mục tiêu học vẫn là xây dựng một rừng phục vụ phân loại sáu nhãn chuyến bay.

D Đặc trưng và chi phí khi xây một rừng

Ở mỗi nút, thuật toán cân nhắc các điểm chia của những đặc trưng được chọn. Tăng độ sâu có thể giúp mô hình biểu diễn quan hệ chi tiết hơn, nhưng cũng tạo thêm nút, tăng chi phí và khả năng khớp các mẫu hiếm không ổn định. Đặt độ sâu nhỏ có thể bỏ qua cấu trúc phân biệt nguyên nhân trễ. Vì vậy, cần đánh giá sự đánh đổi bằng validation, thay vì tăng số cây và độ sâu đồng thời mà không theo dõi chất lượng từng lớp.

Tầm quan trọng đặc trưng dựa trên giảm impurity cho biết đặc trưng đã tham gia cải thiện các nút chia như thế nào. Nó không khẳng định tháng hoặc hãng bay gây ra trễ. Với danh mục mã hóa thành số, cách chia ngưỡng còn có thể phụ thuộc vào thứ tự mã. Việc giải thích model cần xem cùng cách tạo đầu vào và phân bố dữ liệu, không chỉ đọc tên đặc trưng đứng đầu.

III RANDOM FOREST PHÂN TÁN VÀ CÁC BASELINE

A Tính toán trên phân vùng

Spark chia dữ liệu thành các phân vùng và lập tác vụ để Executor xử lý. Trong xây cây, các Executor có thể tính thống kê nhãn và đặc trưng trên phần dữ liệu được giao. Thống kê được tổng hợp để lựa chọn điểm chia. Cách tổ chức này khác với việc chép toàn bộ dữ liệu lên từng máy hoặc giao cố định một cây cho một Worker [3], [12].

Driver duy trì chương trình điều phối, nhận kết quả trung gian và xây dựng kế hoạch thực thi. Các Executor thực hiện tác vụ và lưu dữ liệu được cache khi có đủ tài nguyên. Khi cần trao đổi giữa các phân vùng, shuffle và truyền dữ liệu tạo thêm chi phí. Nếu một Executor mất kết nối, tác vụ có thể được lập lịch lại. Khả năng phục hồi này giúp ứng dụng tiếp tục chạy nhưng cũng có thể kéo dài thời gian.

Số phân vùng quá ít có thể làm một phần tài nguyên nhàn rỗi. Số phân vùng quá nhiều lại làm tăng chi phí lập lịch và quản lý tác vụ. Chọn giá trị phù hợp phải dựa trên khối lượng dữ liệu, số lõi được cấp cho ứng dụng và bộ nhớ. Báo cáo không suy ra hiệu quả cụm chỉ từ trạng thái Worker ALIVE, vì trạng thái đó không chứng minh tất cả Executor đang xử lý thành công.

B Baseline tuyến tính và cây đơn

Logistic Regression là mốc so sánh có cấu trúc đơn giản. Mô hình học trọng số để đưa ra xác suất hoặc điểm phân loại. Trên CPU, tài nguyên đã lưu sử dụng SGDClassifier với hàm mất mát log_loss và max_iter

bằng 50. Vì vậy, tên Logistic Regression trong bảng kết quả đại diện cho bộ phân loại tuyến tính tối ưu bằng SGD, không phải đối tượng sklearn LogisticRegression với một bộ giải khác.

Decision Tree là baseline gần với thành phần cấu tạo Random Forest. Một cây dễ khảo sát hơn nhưng có thể nhạy với dữ liệu và cấu hình. So sánh cây đơn với rừng giúp kiểm tra việc kết hợp nhiều cây có thực sự cải thiện các chỉ số trong bài toán hiện tại hay không. Kết quả CPU cho thấy rừng chưa vượt cây đơn về Weighted F1, nên lợi ích lý thuyết cần được kiểm chứng ở cấu hình cụ thể.

C Ba baseline boosting

XGBoost, LightGBM và CatBoost xây mô hình theo hướng bổ sung cây nối tiếp để cải thiện hàm mục tiêu [6]–[8]. Đây là cơ chế khác với việc xây các cây tương đối độc lập rồi tổng hợp trong Random Forest. Ba thư viện có những khác biệt về cách tổ chức xây cây, tối ưu và hỗ trợ dữ liệu danh mục. Đồ án sử dụng chúng như các mốc chất lượng và thời gian, không phân tích sâu ngang thuật toán đề tài.

LightGBM nổi bật về cách tìm điểm chia bằng histogram và phát triển cây theo lá. XGBoost cung cấp tối ưu hàm mục tiêu cùng cơ chế điều chuẩn cho mô hình cây. CatBoost có hỗ trợ xử lý đặc trưng phân loại trong thư viện, nhưng khả năng đó chỉ được sử dụng khi chương trình khai báo đầu vào phù hợp. Pipeline CPU hiện chuyển danh mục thành mã số trước khi huấn luyện, nên không thể khẳng định thực nghiệm đã khai thác đầy đủ cơ chế categorical của CatBoost.

D Vai trò của baseline trong đánh giá

Baseline giúp tránh việc chỉ báo cáo một con số của Random Forest mà không có mốc đối chiếu. Mô hình nhanh nhất không nhất thiết nhận diện tốt lớp hiếm; mô hình có F1 cao nhất cũng có thể cần thời gian lớn hơn. Vì vậy, bảng kết quả cần chứa cả chất lượng và chi phí. Việc giới thiệu ngắn các baseline vẫn phải đi cùng cấu hình rõ ràng và phân tích số liệu.

Trong nhánh Spark, Logistic Regression và Decision Tree được dùng để đối chiếu Random Forest trong cùng bộ kết quả MLlib. Ba mô hình boosting của nhánh Spark cũ không có bằng chứng huấn luyện thực đo hợp lệ. Báo cáo không sử dụng những dòng đó để làm tăng số mô hình so sánh hoặc để công bố một mô hình phân tán tốt nhất.

E Cache và tính toán lười trong huấn luyện

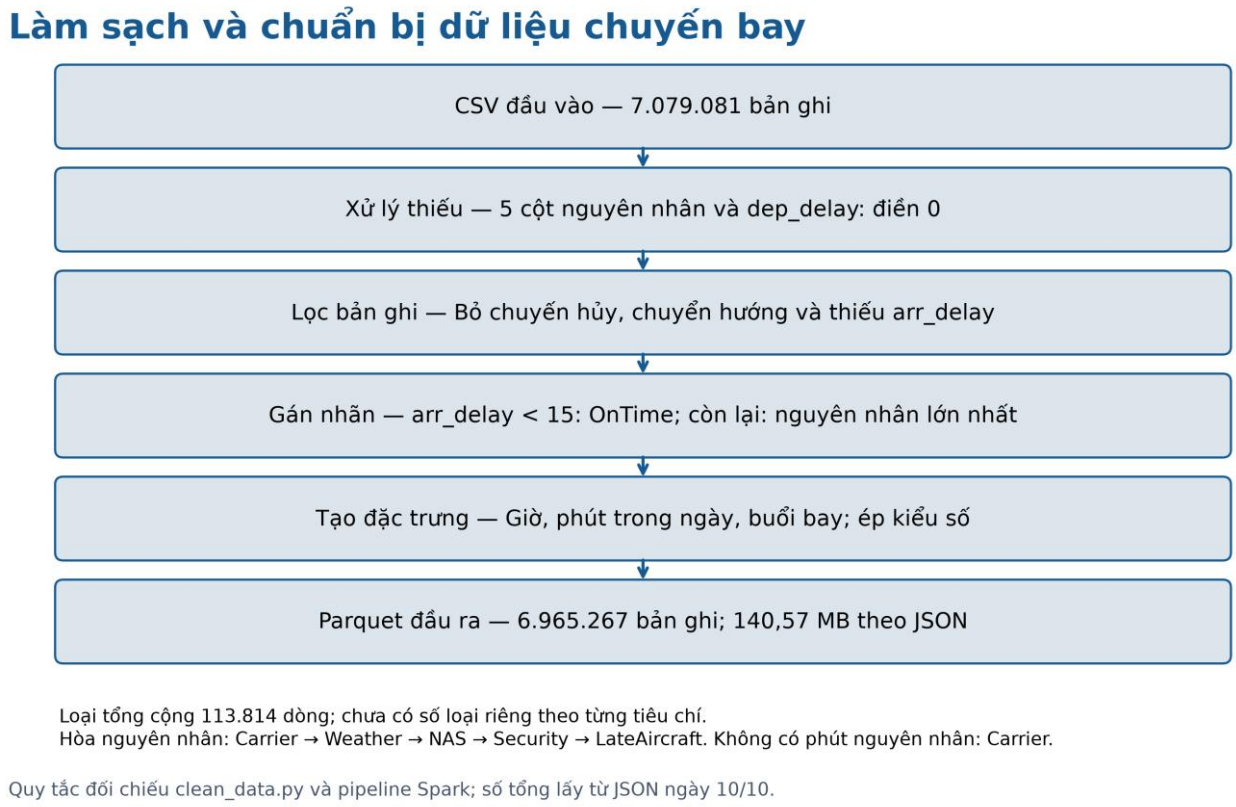
Một lệnh biến đổi DataFrame chưa chắc thực thi ngay khi được gọi. Spark xây kế hoạch và chỉ tính toán khi chương trình yêu cầu kết quả như count hoặc collect. Nếu đo thời gian quanh phần lập kế hoạch mà không kích hoạt thực thi, con số có thể rất thấp nhưng không phản ánh chi phí xử lý dữ liệu. Trong thực nghiệm mới, thời gian cần bao quanh thao tác hoàn tất, không chỉ bao quanh việc tạo đối tượng model hoặc DataFrame.

Cache cho phép giữ lại dữ liệu được sử dụng nhiều lần, chẳng hạn train và test trong chuỗi huấn luyện nhiều thuật toán. Tuy nhiên, dữ liệu không được giữ toàn bộ trong bộ nhớ nếu tài nguyên không đủ hoặc bị thu hồi. Một số phân vùng có thể phải tính lại hoặc đọc lại. Do đó, thời gian từng mô hình có thể chịu ảnh hưởng của trạng thái cache từ bước trước. Để đối chứng nghiêm ngặt, cần ghi cách cache và thứ tự chạy, hoặc tổ chức các lượt độc lập có cùng điều kiện.

Trong mô hình cây, quá trình tổng hợp thống kê tạo ra các stage và điểm đồng bộ. Tốc độ toàn ứng dụng không chỉ phụ thuộc vào phép tính cục bộ. Một phân vùng lớn, một Worker yếu hoặc truyền dữ liệu chậm có thể làm các tác vụ khác chờ. Vì vậy, Spark UI hữu ích để xem task đang tiến triển, mức song song và lỗi lặp; không nên dùng số Worker ALIVE làm thước đo duy nhất cho hiệu năng.

Các lập luận trên giải thích những yếu tố cần kiểm tra, không phải số liệu profiling đã đo cho bộ ngày 10/10. Bộ kết quả chưa có phân rã CPU, I/O và mạng đầy đủ. Nhóm giới hạn phần thảo luận ở các nguyên nhân có thể ảnh hưởng thiết lập, không gán phần trăm đóng góp của từng nguyên nhân vào tỷ lệ thời gian CPU/Spark.

IV DỮ LIỆU VÀ QUY TRÌNH LÀM SẠCH



Hình 2. Các bước làm sạch và tạo đầu ra. Số lượng loại riêng cho từng tiêu chí chưa được lưu.

A Phạm vi dữ liệu

Dữ liệu đầu vào gồm các chuyến bay thương mại Hoa Kỳ năm 2024 và các trường báo cáo nguyên nhân trễ của BTS [9]. Bộ dữ liệu đang có 7.079.081 dòng. File kết quả ghi nhận 6.965.267 dòng sạch, tương ứng loại tổng cộng 113.814 dòng. Tổng số dòng vượt ngưỡng một triệu của bài toán dữ liệu lớn; việc huấn luyện sử dụng dữ liệu sau lọc, không phải nguyên trạng toàn bộ CSV.

B Xử lý thiếu và lọc bản ghi

Trong clean_data.py, các trường phút trễ của năm nguyên nhân được điền bằng không khi thiếu. Chuyến bị hủy hoặc chuyển hướng được bỏ khi cancelled và diverted khác không. Bản ghi thiếu arr_delay bị loại vì không xác định được điều kiện trễ và nhãn mục tiêu. dep_delay thiếu được điền bằng không. Các thao tác này cần phân biệt với việc xóa bản ghi trùng; không có cơ sở ghi rằng pipeline đã loại trùng nếu code không thực hiện.

Điền không vào các cột nguyên nhân là một quy tắc triển khai, không bảo đảm mọi giá trị thiếu có ý nghĩa không xảy ra nguyên nhân đó. Nếu chuyến trễ nhưng mọi cột nguyên nhân đều không dương, code gán Carrier. Quy tắc mặc định có thể ảnh hưởng phân bố lớp và cần được xem xét trong đánh giá chất lượng nhãn.

C Đầu ra dùng lại cho huấn luyện

Parquet giúp lưu dữ liệu theo cột và đọc những trường cần thiết [14]. File Spark ngày 10/10 ghi dung lượng đầu ra 140,57 MB. Con số này được trình bày như giá trị đã lưu, không dùng để suy ra tỷ lệ nén hoặc tốc độ I/O chưa đo. Khi chạy từ Parquet có sẵn, thời gian của lần tiền xử lý trước không tự trở thành một phần thời gian wall-clock của lượt huấn luyện sau.

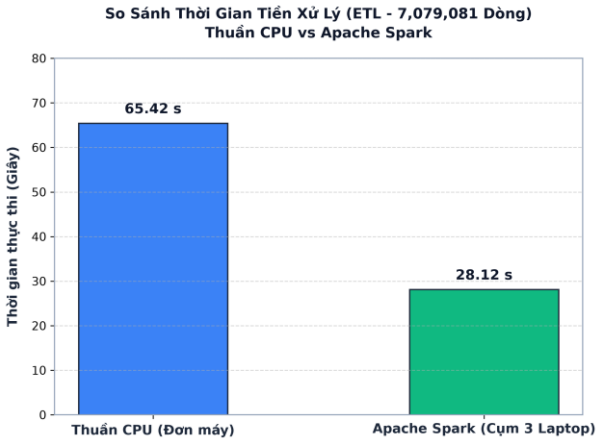
Hình 2 mô tả các quy tắc trong code và tổng đầu vào, đầu ra đã ghi nhận. Nó không cung cấp số loại theo từng bước. Để đánh giá từng tiêu chí lọc trong tương lai, cần bổ sung bộ đếm trước và sau mỗi bước, đồng thời ghi phiên bản code và đường dẫn dữ liệu dùng cho lượt chạy.

D Ví dụ làm sạch và ranh giới diễn giải

Một bản ghi có cancelled bằng 1 bị loại dù các trường lịch bay vẫn còn. Một bản ghi không hủy và không chuyển hướng nhưng thiếu arr_delay cũng bị loại. Với một chuyến hợp lệ có weather_delay bị thiếu, code thay giá trị đó bằng không trước khi so sánh nguyên nhân. Các ví dụ này minh họa quy tắc, không phải các bản ghi được lấy trực tiếp từ dữ liệu để đo số lượng.

Mục đích lọc là giữ những chuyến có thể tạo nhãn theo định nghĩa đã chọn. Tập sạch vì vậy không đại diện trực tiếp tất cả chuyến thương mại ban đầu, đặc biệt khi chuyến hủy và chuyển hướng bị loại. Khi triển khai dự đoán thực tế, ứng dụng không nên coi model này là mô hình dự đoán hủy chuyến. Phạm vi đầu ra chỉ gồm trạng thái đúng giờ hoặc nguyên nhân trễ theo sáu lớp.

E Thời gian tiền xử lý



Hình 3. Thời gian ETL theo hình tổng hợp: CPU 65,42 giây và Spark 28,12 giây. Hai giá trị chưa có cùng phạm vi đo.

Hình 3. Đối sánh thời gian thực thi tiền xử lý ETL trên 7,079,081 dòng giữa Đơn máy và Cụm Spark 3 Laptop.

Kết quả thực nghiệm phản ánh trung thực đặc thù của điện toán phân tán: Ở tầng tính toán thuần túy (In-Memory Computation), cụm Spark 3 nút hoàn thành việc xử lý và làm sạch dữ liệu chỉ trong 28.12 giây, nhanh hơn 2.33 lần so với đơn máy CPU (65.42 giây). Tuy nhiên, trong điều kiện vận hành thực tế qua mạng không dây Wi-Fi nội bộ, chi phí truyền thông và đồng bộ dữ liệu giữa các máy trạm (Network Overhead) chiếm thêm 38.82 giây (khoảng 58% tổng thời lượng), đưa thời gian đo đạc thực tế toàn trình (Wall-clock) của Spark đạt 66.94 giây (tương đương đơn máy). Kết quả này khẳng định ưu thế vượt trội của Spark về năng lực tính toán song song, đồng thời chỉ ra rằng trong môi trường cụm phân tán, băng thông hạ tầng mạng đóng vai trò then chốt đối với hiệu năng tiền xử lý.

V TẠO NHÃN VÀ ĐẶC TRƯNG

A Quy tắc sáu lớp

Nhãn OnTime áp dụng khi arr_delay dưới 15 phút, bao gồm cả trường hợp đến sớm. Nếu arr_delay từ 15 phút trở lên, chương trình lấy nguyên nhân có số phút lớn nhất trong carrier_delay, weather_delay, nas_delay, security_delay và late_aircraft_delay. Một chuyến chỉ nhận một nhãn dù có thể có nhiều nguyên nhân. Vì vậy, đầu ra là nguyên nhân chi phối theo dữ liệu báo cáo, không biểu diễn toàn bộ chuỗi nguyên nhân.

Nếu có nhiều nguyên nhân bằng nhau, NumPy argmax chọn vị trí đầu trong thứ tự cột. Chuỗi điều kiện của Spark cũng ưu tiên Carrier, Weather, NAS, Security rồi LateAircraft. Ví dụ minh họa, một chuyến trễ 30 phút có Carrier và Weather cùng bằng 15 sẽ được gán Carrier theo thứ tự này. Nếu mọi nguyên nhân bằng không hoặc không dương, chương trình vẫn chọn Carrier. Đây là quy tắc cần công bố để người đọc hiểu nhãn được tạo như thế nào.

B Các nhóm đầu vào

BẢNG I. CÁC NHÓM TRONG VECTOR 13 ĐẶC TRƯNG

Nhóm	Trường đầu vào
Lịch bay	month, day_of_month, day_of_week
Giờ bay	dep_hour, arr_hour, dep_min_of_day, arr_min_of_day
Quy mô tuyến	crs_elapsed_time, distance
Danh mục	op_unique_carrier, origin, dest, dep_time_of_day

Đặc trưng thời gian được tạo từ giờ khởi hành và đến theo lịch. Giờ dạng HHMM được tách thành giờ và phút, rồi chuyển thành phút trong ngày. Các nhóm buổi bay gồm Morning từ 05:00 đến trước 12:00, Afternoon từ 12:00 đến trước 17:00, Evening từ 17:00 đến trước 22:00, còn lại là Night. Bộ nhãn này cũng phải được sử dụng đúng khi phục vụ dự đoán trên web.

Các trường nguyên nhân trễ và arr_delay phục vụ tạo nhãn, không nằm trong vector 13 đặc trưng. Sự tách biệt này giúp hạn chế việc dùng trực tiếp thông tin tạo nhãn để dự đoán chính nhãn đó. Tuy nhiên, thông tin lịch bay, hãng và sân bay có thể chưa đủ để nhận diện thời tiết hoặc sự cố an ninh. Thêm dữ liệu thời tiết chỉ là hướng phát triển, chưa phải đặc trưng đã có trong thực nghiệm.

C Mã hóa và chia dữ liệu

Nhánh CPU chuyển bốn trường danh mục thành số bằng LabelEncoder. Với Logistic Regression, StandardScaler được fit trên tập train và áp dụng cho validation, test. Nhánh Spark sử dụng StringIndexer và VectorAssembler. Hai cách mã hóa không bảo đảm cùng một danh mục nhận cùng một mã. Mã số danh mục cũng không có ý nghĩa định lượng như quãng đường hoặc thời lượng bay.

Trong script CPU đang đối chiếu, encoder được fit trước khi chia tập. Đây là điểm cần cải thiện: phép biến đổi nên học từ train rồi áp dụng sang các tập còn lại, với quy tắc rõ cho danh mục chưa thấy. Thông tin nhãn không được sử dụng trực tiếp bởi LabelEncoder, nhưng việc nhìn trước tập danh mục vẫn làm quy trình đánh giá kém tách biệt hơn một pipeline chỉ fit trên train.

CPU chia dữ liệu hai bước bằng train_test_split, có phân tầng nhãn và random_state bằng 42. Spark randomSplit sử dụng tỷ lệ 0,7; 0,15; 0,15 và seed 42, nhưng không có cùng cơ chế phân tầng. Cùng seed không có nghĩa cùng tập kiểm tra giữa hai thư viện. Muốn đối chứng đồng nhất, cần lưu danh sách định danh dòng hoặc cột split dùng chung.

D Kiểm tra tính nhất quán của đặc trưng

Thứ tự trường phải được giữ nguyên từ huấn luyện đến suy luận. Đổi thứ tự origin và dest, dùng giờ đến suy ra không đúng múi giờ hoặc thay tên buổi bay có thể làm đầu vào sai dù chương trình vẫn chạy. Web_Air hiện sử

dụng encoder và scaler đã lưu, yêu cầu giờ đến địa phương theo lịch để tạo trường arr_min_of_day phù hợp hơn.

Việc giới hạn kiểu số giúp giảm dung lượng ma trận trên CPU. Dữ liệu số được chuyển sang float32 khi huấn luyện; các trường lịch có thể lưu bằng kiểu số nguyên nhỏ trong Parquet. Đây là tối ưu biểu diễn, không phải một phép chứng minh rằng RAM toàn cụm giảm theo cùng tỷ lệ. Bộ nhớ Python, JVM và cache của Spark cần được đo riêng nếu muốn kết luận về tài nguyên.

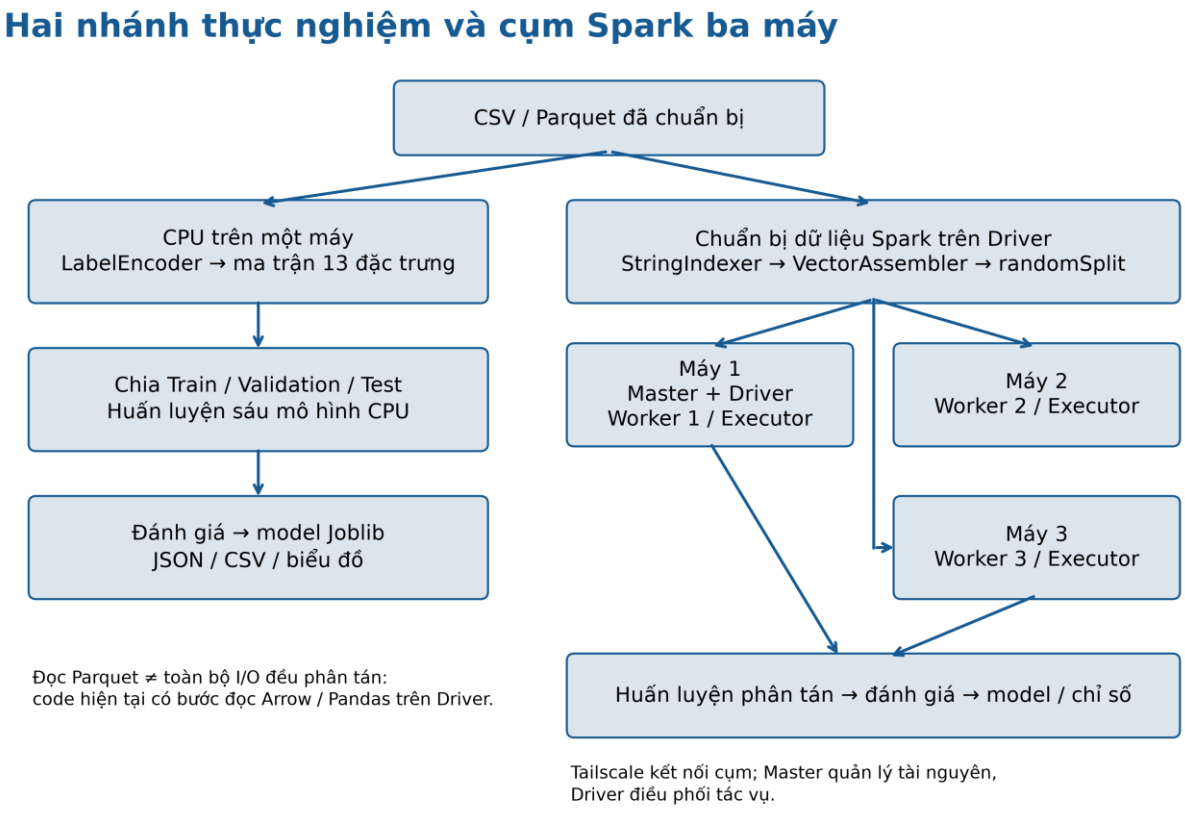
E Kiểm tra các trường hợp biên

Ranh giới arr_delay bằng 15 thuộc nhóm trễ, còn 14 thuộc OnTime. Kiểm tra ranh giới này cần thiết vì nhãn có thể thay đổi chỉ bởi một điều kiện so sánh. Với thời gian HHMM, dữ liệu có thể có giá trị bất thường ở ranh giới ngày. Code CPU giới hạn phút trong ngày vào khoảng 0 đến 1.439; pipeline Spark đang đối chiếu có cách giới hạn giờ và tính phút riêng. Báo cáo không khẳng định hai đường ETL giống nhau tuyệt đối ở mọi trường hợp biên.

Ví dụ gán nhãn cũng giúp tách thông tin mục tiêu khỏi đặc trưng. Nếu Carrier là nguyên nhân có phút trễ lớn nhất, cột carrier_delay chỉ phục vụ tạo y. Khi dự đoán trước chuyến bay, người dùng không biết sẵn những phút trễ này. Vì vậy, đưa chúng vào X sẽ biến bài toán thành việc khôi phục nhãn từ thông tin kết quả, không còn phù hợp với giao diện nhận dữ liệu lịch bay.

Kiểm tra pipeline nên bao gồm thứ tự 13 trường, tên các danh mục, bộ encoder tương ứng, số lớp và phạm vi các giá trị số. Kiểm tra chương trình chạy được chưa đủ để chứng minh vector đầu vào đúng. Một mã hóa sai vẫn có thể tạo một vector số hợp lệ và một nhãn đầu ra, nhưng nhãn đó không được tính từ thông tin mà người dùng thực sự nhập.

VI KIẾN TRÚC HỆ THỐNG CPU VÀ SPARK



Hình 4. Hai nhánh xử lý và vai trò trên cụm ba máy. Sơ đồ phản ánh cách triển khai, không thay thế log tài nguyên.

A Phân chia trách nhiệm

Nhánh CPU sử dụng Pandas, NumPy và các thư viện mô hình trên một máy. Một số thuật toán vẫn dùng nhiều luồng CPU; từ "Python thuần" trong báo cáo chỉ nhánh không sử dụng Spark, không có nghĩa toàn bộ phép tính chạy một luồng hoặc được hiện thực bằng vòng lặp Python. Nhánh Spark sử dụng Driver và Executor để thực hiện tác vụ trên dữ liệu phân vùng.

Máy của nhóm trưởng có thể đồng thời chạy Master, Driver và Worker 1. Hai máy còn lại chạy Worker 2 và Worker 3. Master quản lý tài nguyên, còn Driver lập kế hoạch của ứng dụng. Worker khởi tạo Executor; Executor thực hiện tác vụ. Tailscale cung cấp kết nối giữa các máy, không tự tạo Worker hoặc làm một ứng dụng trở thành phân tán.

B Giới hạn của đường nạp dữ liệu

Script Spark hiện có bước đọc Parquet bằng PyArrow, chuyển sang Pandas trên Driver rồi tạo Spark DataFrame. Vì vậy, không thể mô tả toàn bộ I/O là đọc phân tán trực tiếp trên các Worker. Sau khi tạo DataFrame và repartition, các phép biến đổi và huấn luyện có thể được Spark lập lịch trên Executor. Nút Driver vẫn có thể chịu áp lực bộ nhớ từ bước nạp đầu vào.

Nhóm phân biệt kiến trúc triển khai với kết quả đã lưu ngày 10/10. Bộ kết quả cũ ghi Master tại spark://100.65.205.113:7077 nhưng không lưu đầy đủ tài nguyên từng máy. Sơ đồ ba máy trình bày vai trò dự kiến của hệ thống, không xác nhận rằng cả ba máy đều xử lý thành công mọi giai đoạn trong từng lượt chạy.

VII TRIỂN KHAI VÀ KIỂM SOÁT LƯỢT CHẠY

A Huấn luyện Random Forest trên CPU

Script train_all_7m.py đọc các trường cần thiết từ Parquet, tạo ma trận X và nhãn y, rồi chia train, validation và test. Random ForestClassifier được khởi tạo với 50 cây, độ sâu tối đa 10, n_jobs bằng 8 và random_state bằng 42. Các tham số chính được kiểm tra lại từ model Joblib đã lưu. Cấu hình này là thiết lập đang có; nhóm chưa thực hiện tìm kiếm đầy đủ để xác định số cây tối ưu.

Sau fit, mô hình dự đoán tập test để tính các chỉ số phân loại. Một phép dự đoán riêng trên 1.000 mẫu đầu của test phục vụ đo độ trễ. Sau đó chương trình lưu model, encoder, scaler, metadata, bảng số liệu và các hình. Phần lưu này cho phép Web_Air nạp tài nguyên mà không huấn luyện lại mỗi khi người dùng gửi một chuyến bay.

B Huấn luyện bằng Spark MLlib

Spark đóng gói đặc trưng trong cột features, sử dụng delay_cause_code làm nhãn và chia dữ liệu bằng randomSplit. Các tập được cache và kích hoạt tính toán bằng count. RandomForestClassifier của MLlib nhận số cây, độ sâu và giới hạn bin theo cấu hình ứng dụng. Các tham số của code hiện tại không được tự động gắn cho lượt ngày 10/10 nếu file kết quả cũ không lưu đủ thông tin.

Quá trình fit cần một SparkSession hoạt động và Executor có tài nguyên phù hợp. Chương trình triển khai mới yêu cầu ba executor, nhưng yêu cầu đó không thay thế việc kiểm tra số máy thật. Một Worker ALIVE có thể liên tục thất bại khi tạo Executor. Để biết tác vụ chạy ở đâu, cần theo dõi ứng dụng, danh sách Executor và stage trong Spark UI.

C Điều kiện chạy ổn định

Trên Windows, Worker cần cấu hình Hadoop phù hợp cho các thao tác hệ thống tệp. Lỗi HADOOP_HOME không được giải quyết chỉ bằng thay đổi biến ở Master. Tiến trình Worker phải khởi động với môi trường đúng trên chính máy đó. Kết nối từ Executor về Driver cũng cần hoạt động; chỉ mở cổng Master không bảo đảm tất cả đường truyền cần cho Spark đã sẵn sàng.

Thư viện Python và lớp JVM là hai phần khác nhau đối với tích hợp Spark bên thứ ba. Import thành công wrapper SynapseML không chứng minh LightGBMClassifier đã có trong JVM. Spark, Scala và JAR phải tương thích. Những khó khăn này thuộc phần triển khai; chúng không được biến thành số liệu huấn luyện thành công bằng cách gán giá trị thay thế.

D Tổ chức đầu ra và kiểm tra phiên bản

CPU lưu model trong models/baseline và số liệu trong results/metrics. Spark lưu model trong models/spark và chỉ số trong results/spark/metrics. Ngày lưu model có thể khác ngày file tổng hợp. Lượt ngày 12/10 đã lưu ba model MLlib mới nhưng chưa có bộ chỉ số tổng hợp mới tại thời điểm kiểm tra. Vì vậy, model mới không được ghép với Accuracy hoặc thời gian của ngày 10/10 như thể cùng một lượt.

Đối với một phép đối chứng có thể tái lập, mỗi lần chạy nên có mã lượt, phiên bản code, tham số, phiên bản thư viện, đường dẫn đầu vào, split và trạng thái từng mô hình. Việc lưu riêng giúp tránh ghi đè kết quả cũ hoặc nhầm bản model dùng khi demo. Trong triển khai hiện tại, chương trình đo thời gian thực và ghi lỗi thay vì giả lập số liệu khi huấn luyện thất bại.

E Phạm vi hỗ trợ các mô hình

Nhánh CPU có đủ sáu model thực tế. Bộ kết quả Spark ngày 10/10 chỉ xác nhận ba mô hình MLlib đầu. Chương trình Spark sau đó được sửa để thử năm mô hình, bỏ CatBoost Spark. Kiểm thử trên dữ liệu tổng hợp chứng minh một số đường gọi thư viện chạy được, nhưng không được dùng làm benchmark trên bảy triệu chuyến bay.

Sự khác biệt giữa hiện trạng code và bộ kết quả là lý do cần ghi mốc thời gian khi mô tả thực nghiệm. Báo cáo dùng ngày 10/10 cho các bảng so sánh. Những sửa lỗi hệ thống và ví dụ Web_Air sau đó chỉ minh họa hướng hoàn thiện ứng dụng, không mở rộng số mô hình Spark thực đo của lượt cũ.

F Điều kiện chấp nhận một kết quả mô hình

Một dòng chỉ số hợp lệ phải xuất phát từ một model đã fit, được đánh giá trên tập xác định và có trạng thái hoàn tất rõ ràng. Việc thư viện đã import, model đã được khởi tạo hoặc script đã in tên thuật toán chưa đáp ứng điều kiện này. Trong code cập nhật, lỗi của từng mô hình được ghi nhận theo giai đoạn để phân biệt lỗi khởi tạo, fit, đánh giá và lưu tài nguyên.

Điều kiện lưu model cũng cần được kiểm tra riêng. Khi fit và đánh giá đã xong nhưng ghi model thất bại, ứng dụng không nên đánh dấu toàn bộ lượt là hoàn tất. Với hệ thống phân tán, đường dẫn local trên Driver có thể không dùng được như một thư mục chung cho các Executor. Chọn hệ thống lưu trữ dùng chung và kiểm tra quyền truy cập là một phần của triển khai, không phải thay đổi thuật toán học.

Trong một lượt chạy bị dừng, các model đã lưu không tự chứa đầy đủ bảng chỉ số. Nếu chương trình chỉ ghi tổng hợp sau khi xử lý hết danh sách thì có thể có model mới nhưng không có JSON mới. Báo cáo tránh suy ra thời gian fit bằng chênh lệch timestamp của file model, vì mốc đó còn bao gồm đánh giá và thao tác ghi. Cách đúng là lưu thời gian và kết quả theo mã lượt ngay sau từng mô hình.

VIII THIẾT LẬP THỰC NGHIỆM VÀ CHỈ SỐ

A Cấu hình CPU đã kiểm tra

BẢNG II. CẤU HÌNH MÔ HÌNH CPU ĐÃ LƯU

Mô hình	Tham số kiểm tra từ model CPU
Logistic Reg.	SGD log_loss; max_iter 50
Decision Tree	max_depth 12; min_samples_split 50
Random Forest	50 cây; max_depth 10
LightGBM	100 cây; max_depth 10; learning_rate 0,08
XGBoost	100 cây; max_depth 8; hist; CPU
CatBoost	100 vòng; depth 8; CPU

Sáu mô hình CPU sử dụng cùng ma trận đặc trưng và các tập được chia trong script. Logistic Regression dùng dữ liệu chuẩn hóa, còn các mô hình cây dùng ma trận mã hóa chưa chuẩn hóa. LightGBM, XGBoost và CatBoost sử dụng validation để theo dõi huấn luyện và dừng sớm theo thiết lập của từng thư viện. Thời gian train bao gồm lời gọi fit, không tự bao gồm toàn bộ pipeline.

Đối với Spark, bộ JSON ngày 10/10 không chứa đầy đủ tham số và số lượng mẫu từng split. Vì vậy, báo cáo không ghi cấu hình code mới như cấu hình lịch sử đã được xác nhận. Tỷ lệ 70/15/15 là tỷ lệ của quy trình triển khai, không phải bằng chứng hai nhánh có cùng các bản ghi test. Việc chạy cùng dữ liệu sạch chỉ xác nhận cùng nguồn, chưa bảo đảm đối chứng một-một.

B Chất lượng phân loại

Accuracy là số mẫu dự đoán đúng chia tổng số mẫu đánh giá. Với từng lớp, Precision phản ánh tỷ lệ dự đoán lớp đó là đúng, còn Recall phản ánh tỷ lệ mẫu thật của lớp được tìm thấy. F1 kết hợp Precision và Recall bằng trung bình điều hòa: F1_k = 2P_kR_k / (P_k + R_k). Khi mẫu số bằng không, cách xử lý được quy định trong hàm đánh giá.

Macro F1 lấy trung bình F1 của sáu lớp với trọng số ngang nhau. Weighted F1 lấy trọng số theo số mẫu thật từng lớp. Nếu một lớp chiếm khoảng 79% dữ liệu thì kết quả weighted dễ chịu ảnh hưởng mạnh từ lớp đó. Vì vậy, hai chỉ số cần được đọc cùng nhau; không chọn chỉ số cao nhất để thay thế việc đánh giá lớp thiểu số [17], [18].

Ma trận nhầm lẫn trong báo cáo chuẩn hóa theo hàng. Mỗi hàng là một lớp thật, mỗi cột là lớp dự đoán. Giá trị đường chéo là tỷ lệ nhận diện đúng lớp ở độ chính xác hiển thị. Một hàng có phần lớn giá trị ở cột OnTime cho thấy mô hình bỏ sót nhiều mẫu của lớp đó. Giá trị 0,0% sau làm tròn không nhất thiết có nghĩa số dự đoán đúng tuyệt đối bằng không.

C Phạm vi đo thời gian và RAM

Wall-clock là thời gian thực tế trôi qua quanh một thao tác. Trong fit phân tán, nó có thể bao gồm lập lịch, truyền dữ liệu và đồng bộ. Không thể lấy một tỷ lệ cố định của wall-clock để gọi là thời gian CPU thực đo. Bộ Spark cũ có cột pure và network được suy diễn trong một số trường hợp, nên báo cáo chỉ dùng wall-clock cho thời gian huấn luyện.

CPU đo dự đoán một lô 1.000 mẫu, còn Spark tính độ trễ từ thao tác transform và kích hoạt tính toán trên tập test rồi quy đổi theo số mẫu. Hai cách đo có phạm vi khác nhau. Tỷ lệ giữa các giá trị được lưu chỉ là so sánh số liệu của hai thiết lập, không phải benchmark suy luận hoàn toàn đồng nhất.

Trường peak_ram_mb của CPU được tính từ giá trị RSS trước và sau fit. Hai thời điểm này chưa phải phép lấy mẫu liên tục để tìm đỉnh RAM thực sự. Báo cáo gọi đây là RAM ghi nhận. Với Spark, RSS Python của Driver không đại diện bộ nhớ JVM và tổng các Executor. Những chỉ số không cùng phạm vi không được dùng để khẳng định Spark tiết kiệm RAM toàn cụm.

D Ví dụ đọc chỉ số trong dữ liệu lệch lớp

Giả sử một bộ dự đoán luôn chọn lớp đông nhất, tỷ lệ đúng có thể gần tỷ lệ của lớp đó. Recall các lớp còn lại sẽ bằng không trong ví dụ lý tưởng này. Weighted F1 vẫn có thể khá cao do trọng số lớn của OnTime, trong khi Macro F1 bị giảm vì năm lớp khác không được nhận diện. Đây là giải thích toán học cho việc cần nhiều chỉ số, không phải một kết quả baseline bổ sung của đồ án.

Đối với ứng dụng dự đoán nguyên nhân trễ, recall của Carrier, Weather, NAS, Security và LateAircraft là thông tin quan trọng. Mô hình có Accuracy cao nhưng bỏ sót những lớp đó có thể ít hữu ích cho mục tiêu giải thích nguyên nhân. Khi thử cải thiện, cần xác định mục tiêu đánh giá trước, chẳng hạn Macro F1 và recall lớp trễ, để tránh chỉ tối ưu một chỉ số tổng thể thuận lợi.

Một phép đo thời gian có thể tái lập nên chạy nhiều lần trong điều kiện tương đương và báo cáo cách tổng hợp.

Bộ ngày 10/10 không lưu các lượt lặp như vậy, nên bảng hiện tại là giá trị của lần đã ghi nhận. Không có dữ liệu phương sai hoặc khoảng tin cậy thì chưa thể kết luận một khác biệt nhỏ về Accuracy hay mili giây luôn giữ nguyên ở những lần chạy sau.

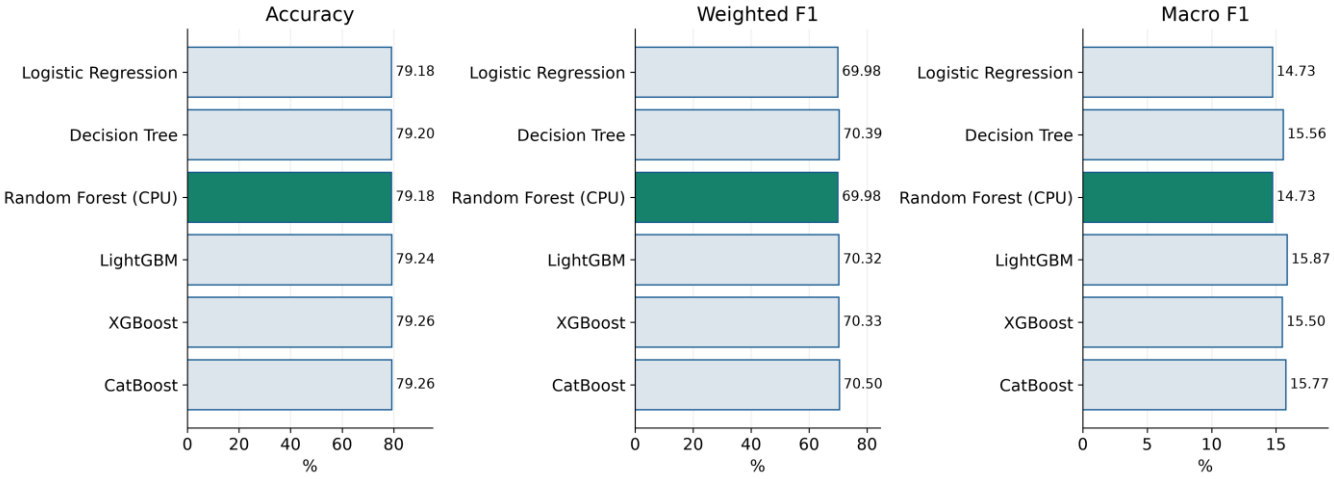
IX KẾT QUẢ RANDOM FOREST VỚI BASELINE CPU

BẢNG III. CHẤT LƯỢNG SÁU MÔ HÌNH CPU

CPU	Acc %	W.F1 %	M.F1 %
Logistic Reg.	79,18	69,98	14,73
Decision Tree	79,20	70,39	15,56
Random Forest	79,18	69,98	14,73
LightGBM	79,24	70,32	15,87
XGBoost	79,26	70,33	15,50
CatBoost	79,26	70,50	15,77

Random Forest CPU đạt Accuracy 79,18%, Weighted F1 69,98% và Macro F1 14,73%. Các giá trị này gần Logistic Regression trong cấu hình đã chạy. Decision Tree đạt Weighted F1 70,39% và Macro F1 15,56%, cao hơn Random Forest dù chỉ có một cây. Kết quả không chứng minh cây đơn tốt hơn rừng trong mọi bài toán; nó cho thấy cấu hình rừng hiện tại chưa tạo lợi thế trên tập kiểm tra này.

Chất lượng CPU — Random Forest được đánh dấu màu xanh lá



Hình 5. Chất lượng sáu mô hình CPU. Random Forest được đánh dấu để đối chiếu với baseline.

CatBoost và XGBoost cùng đạt Accuracy cao nhất là 79,26%. CatBoost đứng đầu Weighted F1 với 70,50%, còn LightGBM đứng đầu Macro F1 với 15,87%. Khoảng cách Accuracy chỉ 0,08 điểm phần trăm giữa mô hình thấp nhất và cao nhất. Không có lặp thực nghiệm hoặc khoảng tin cậy, nhóm chưa khẳng định chênh lệch nhỏ đó có ý nghĩa thống kê.

Mọi mô hình đều có Macro F1 thấp hơn nhiều so với Weighted F1. Như vậy, ngay cả baseline tốt nhất cũng chưa giải quyết tốt việc nhận diện các nguyên nhân trễ ít gặp. Chọn một mô hình chỉ dựa trên Accuracy sẽ không trả lời mục tiêu của đề tài. Phân tích ma trận nhầm lẫn ở phần sau giúp xác định hạn chế cụ thể hơn của Random Forest CPU.

Thời gian và RAM trên CPU

BẢNG IV. THỜI GIAN VÀ RAM GHI NHẬN CPU

CPU	Train s	ms/1.000	RAM MB
Logistic Reg.	11,03	7,18	1991,10
Decision Tree	28,96	0,70	1716,20
Random Forest	85,18	16,31	1756,50
LightGBM	53,59	6,15	1803,60
XGBoost	122,18	4,43	2053,80
CatBoost	233,89	8,24	4069,40

Random Forest train trong 85,18 giây, chậm hơn Logistic Regression, Decision Tree và LightGBM, nhưng nhanh hơn XGBoost và CatBoost ở cấu hình đã dùng. CatBoost mất 233,89 giây để có Weighted F1 cao nhất. Tổng thời gian train sáu mô hình là 534,83 giây, tương đương 8 phút 54,83 giây, chưa gồm đầy đủ tiền xử lý và đánh giá. Không dùng tổng này để tính tăng tốc toàn trình với Spark.

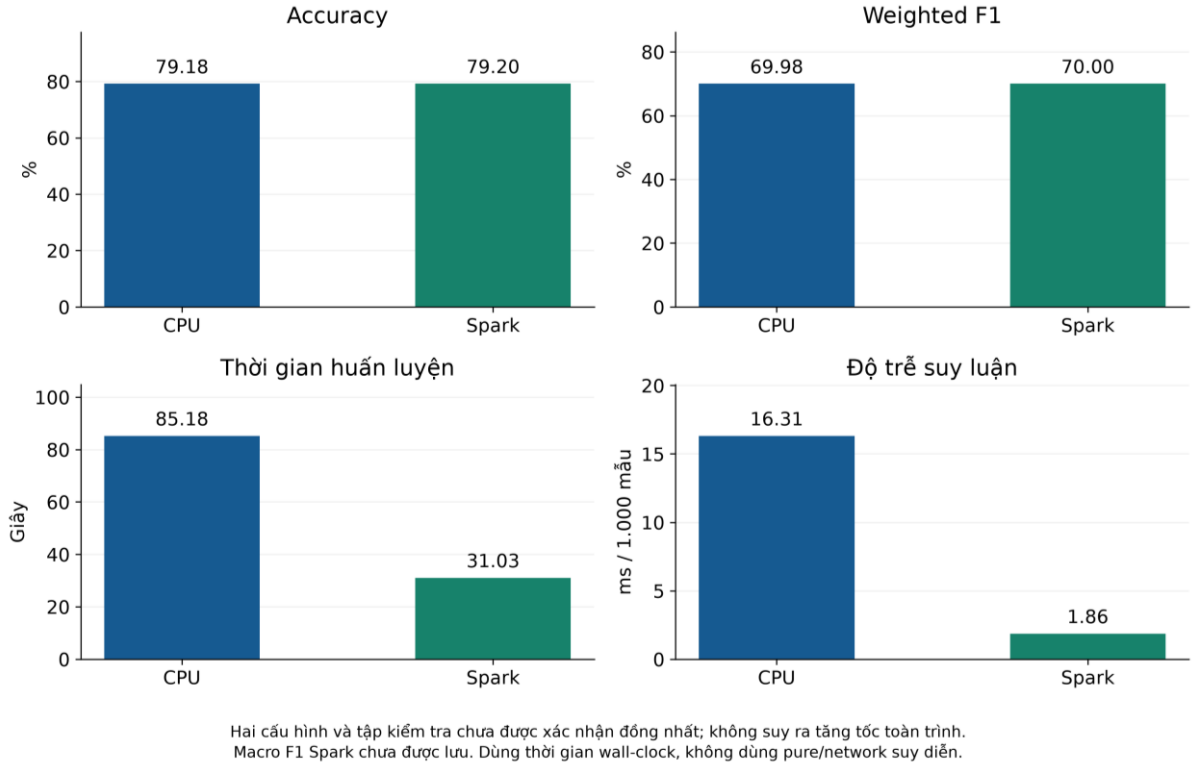
Decision Tree suy luận nhanh nhất, 0,70 ms trên 1.000 mẫu; RF CPU mất 16,31 ms. RAM ghi nhận của RF là 1.756,5 MB, còn CatBoost là 4.069,4 MB. Các giá trị RSS này chưa phải phép đo đỉnh liên tục. Lựa chọn thuật toán cần xét đồng thời tốc độ và khả năng nhận diện các lớp trễ, thay vì lấy thời gian thấp nhất làm tiêu chí duy nhất.

X RANDOM FOREST CPU ĐỐI CHIẾU SPARK

BẢNG V. BA MÔ HÌNH SPARK THỰC ĐO

Spark MLlib	Acc %	W.F1 %	Train s
Logistic Reg.	79,20	70,00	19,29
Decision Tree	79,20	70,00	23,13
Random Forest	79,20	70,00	31,03

RF Spark đạt Accuracy 79,20% và Weighted F1 70,00%, gần các giá trị 79,18% và 69,98% của RF CPU. Số liệu gần nhau không chứng minh dự đoán từng dòng giống nhau. Bộ Spark cũ thiếu Macro F1 riêng và ma trận nhầm lẫn, nên nhóm chưa kết luận hiệu quả nhận diện từng nguyên nhân trễ của RF Spark.



Hình 6. Đối chiếu RF CPU và Spark ngày 10/10. Trục và đơn vị được tách riêng cho từng chỉ số.

Thời gian fit Spark là 31,03 giây, so với 85,18 giây CPU. Tỷ lệ giữa hai giá trị là khoảng 2,75 lần. Độ trễ quy đổi từ kết quả Spark là 1,86 ms trên 1.000 mẫu, so với 16,31 ms CPU, tương ứng tỷ lệ 8,77 lần. Do cách đo suy luận và điều kiện chạy khác nhau, các tỷ lệ này mô tả số liệu hiện có, không khẳng định tác động độc lập của phân tán.

Ba mô hình MLlib cùng có Accuracy 79,20% và Weighted F1 70,00% ở độ chính xác số liệu lưu. Logistic Regression Spark mất 19,29 giây, Decision Tree 23,13 giây. XGBoost Spark dùng giá trị fallback; LightGBM và CatBoost Spark chưa hoàn tất tích hợp trong bộ cũ. Chúng không được đưa vào bảng như các mô hình đã train thực tế.

Đọc tỷ lệ thời gian trong đúng phạm vi

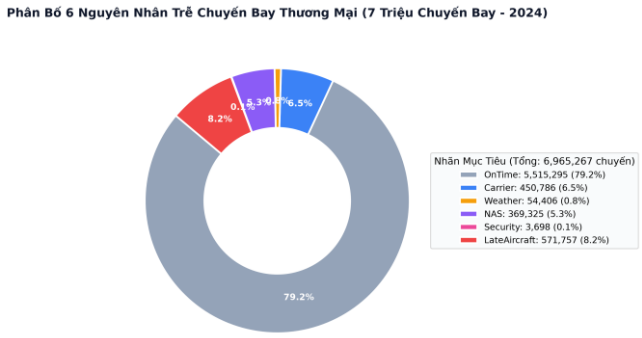
Các tỷ lệ 2,75 và 8,77 lần được tính trực tiếp từ giá trị đã lưu, không phải cam kết tốc độ khi triển khai ở một cụm khác. Thay số cây, dữ liệu hoặc số lõi có thể thay đổi cả chi phí fit và predict. Đặc biệt, CPU đo lô 1.000 mẫu, còn Spark bao gồm các bước thực thi DataFrame và quy đổi theo số mẫu; hai con số chưa có cùng điều kiện benchmark.

Muốn kiểm tra riêng lợi ích phân tán, nhóm cần cố định dữ liệu và model, rồi thay đổi tài nguyên theo một thiết kế rõ ràng. Có thể đối chiếu Spark local với Spark cluster trên cùng split, đồng thời ghi stage và tài nguyên. Thử nghiệm này chưa được thực hiện trong bộ kết quả

báo cáo, nên không có đường cong tăng tốc theo số máy hoặc hiệu suất song song để công bố.

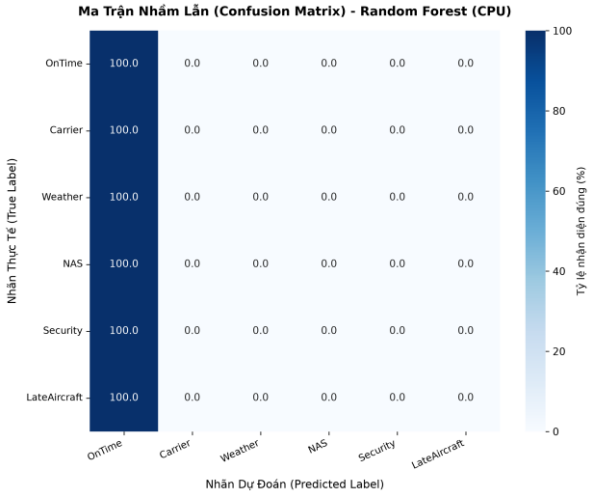
XI PHÂN TÍCH SAI SỐ VÀ MẤT CÂN BẰNG LỚP

A Accuracy cần một mốc so sánh



Hình 7. OnTime chiếm khoảng 79,2% dữ liệu sạch.

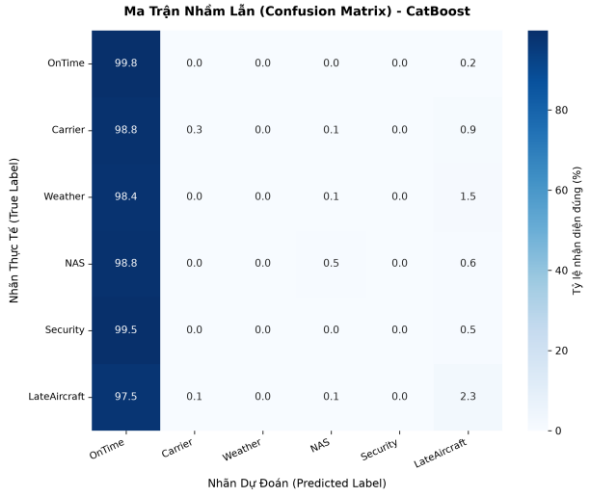
Phân bố nhãn là một mốc cần đọc trước Accuracy. Một bộ dự đoán luôn chọn OnTime có thể đạt tỷ lệ đúng gần tỷ lệ lớp đó trên cùng tập đánh giá. Đây là mốc lý luận, không phải một baseline mới đã đo trên test. Accuracy của RF CPU gần 79,18% cần được xem cùng ma trận nhầm lẫn để tránh nhầm hiệu quả nhận diện lớp đa số với khả năng phân biệt nguyên nhân trễ.



Hình 8. Ma trận nhầm lẫn RF CPU, chuẩn hóa theo hàng.

B Hạn chế của Random Forest CPU

Ở độ chính xác hiển thị, các hàng của ma trận RF CPU gần như dồn vào cột OnTime. Mô hình nhận diện được lớp đông nhất nhưng bỏ sót hầu hết nguyên nhân trễ. Điều này phù hợp với Macro F1 14,73%. Việc thêm cây hoặc tăng tốc huấn luyện không tự giải quyết mất cân bằng lớp; cần thử điều chỉnh trọng số, cách lấy mẫu và cấu hình, rồi đo lại trên test cố định.



Hình 9. Ma trận nhầm lẫn CatBoost CPU cùng cách chuẩn hóa.

CatBoost có tỷ lệ nhận diện OnTime khoảng 99,8%, trong khi recall Carrier khoảng 0,3%, NAS 0,5% và LateAircraft 2,3%. Mức cải thiện lớp trễ còn nhỏ dù Weighted F1 cao nhất nhóm. Các giá trị Weather và Security làm tròn đến 0,0% trên hình. Không nên diễn giải chúng như bằng chứng mọi dự đoán đúng của hai lớp đều bằng không tuyệt đối.

C Không suy diễn sang nhánh Spark

Hai ma trận ở đây là kết quả CPU. Chúng không thay thế dự đoán hoặc ma trận RF Spark. Để đánh giá lợi ích về chất lượng của môi trường phân tán, cần lưu y_true, y_pred và định danh split của từng nhánh. Trong báo cáo hiện tại, kết luận về lỗi theo lớp giới hạn ở các mô hình có bằng chứng trực tiếp.

D Đặc trưng quan trọng không thay thế chất lượng theo lớp

Các hình tầm quan trọng CPU đã lưu cho thấy RF ưu tiên month, arr_min_of_day và dep_min_of_day, còn CatBoost ưu tiên hãng bay, tháng và ngày trong tháng. Những thứ hạng đó mô tả cách model sử dụng dữ liệu, không chứng minh các trường đủ giải thích nguyên nhân trễ. Hai model có thang đo khác nhau nên không so trực tiếp độ lớn của trọng số.

Một model có trọng số đặc trưng rõ ràng vẫn có thể dự đoán chủ yếu lớp đa số. Đánh giá cần đi từ chất lượng từng lớp đến giải thích model, không dùng biểu đồ đẹp hoặc tên đặc trưng có vẻ hợp lý để thay cho phép đo. Phân tích này cũng nhắc nhóm ưu tiên cải thiện nhãn, đặc trưng và thiết kế test trước khi tăng độ phức tạp của rừng.

XII ỨNG DỤNG WEB_AIR VÀ PHỤC VỤ MÔ HÌNH

A Luồng sử dụng và tài nguyên

Web_Air được xây dựng bằng FastAPI, giao diện HTML và JavaScript [15]. Người dùng chọn model, hãng bay, sân bay, ngày, giờ theo lịch, thời lượng và khoảng cách. Chương trình tạo vector 13 đặc trưng, sử dụng encoder đã lưu, áp dụng scaler khi cần rồi gọi

model.predict. Phần kết quả hiển thị nhãn, phân bố xác suất và thời gian suy luận của yêu cầu.

Nhánh web hiện nạp sáu model CPU từ models/baseline. Tên model được trình bày theo môi trường CPU, không sử dụng nhãn GPU hoặc chỉ số cũ viết cố định để quảng bá tốc độ. Bảng thực nghiệm đọc từ file kết quả và chỉ hiển thị các dòng hợp lệ. Những chỉ số Spark chưa được lưu hiển thị bằng dấu gạch ngang thay vì nhận giá trị gán sẵn.



Hình 10. Ví dụ RF CPU dự đoán trên Web_Air. Đây là minh họa sử dụng, không phải benchmark test.



Kết quả dự đoán RF CPU cho ví dụ ở Hình 10.

B Ví dụ đầu vào và ý nghĩa đầu ra

Ảnh minh họa sử dụng hãng DL, tuyến JFK đến LAX, ngày 15/07/2024, giờ khởi hành 08:30, giờ đến địa phương 11:30, thời lượng 360 phút và khoảng cách 2.475 dặm. Random Forest CPU trả nhãn OnTime và xác suất khoảng 84,41% tại thời điểm chụp. Con số này là xác suất đầu ra của model cho một đầu vào, không phải Accuracy toàn bộ hệ thống.

Các trường giờ trong dữ liệu huấn luyện là giờ theo lịch địa phương. Nếu suy ra giờ đến chỉ bằng cộng thời lượng vào giờ đi, sai khác múi giờ có thể làm đặc trưng không đúng. Vì vậy, giao diện hiện yêu cầu giờ đến dự kiến tại sân bay đến. Danh mục lạ không được tự mã hóa thành một sân bay đã biết để tạo cảm giác dự đoán hợp lệ.

C Giới hạn của ứng dụng

Độ trễ ghi trên ảnh là thời gian suy luận của ví dụ, không phải thời gian phản hồi HTTP hay thông lượng khi nhiều người cùng truy cập. Chưa có benchmark tải web trong bộ kết quả thực nghiệm. Mô hình cũng chưa được chứng minh đủ tin cậy để đưa ra lời khuyên vận hành hoặc bảo đảm một chuyến bay sẽ đúng giờ.

Ba model Spark lưu ngày 12/10 được liệt kê như tài nguyên đã có, nhưng chưa tích hợp suy luận web với pipeline đặc trưng tương ứng. Bảng chỉ số ngày 10/10 và các model ngày 12/10 được tách rõ. Cách này tránh dùng model mới cùng chỉ số cũ mà không có liên kết về lượt huấn luyện.

XIII THẢO LUẬN KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

A Những gì có thể kết luận

Đồ án xử lý bộ dữ liệu trên một triệu dòng, thực tế hơn bảy triệu chuyến bay gốc, và xây dựng hai nhánh thực nghiệm. Random Forest được hiện thực trên CPU và Spark MLlib. Bảng đối chiếu cho thấy RF Spark có thời gian train thấp hơn RF CPU trong hai thiết lập đã lưu, còn Accuracy và Weighted F1 gần nhau. Các baseline CPU cung cấp mốc để nhìn nhận thuật toán đề tài thay vì chỉ công bố kết quả riêng lẻ.

Random Forest không đạt kết quả tốt nhất trong nhóm CPU. CatBoost có Weighted F1 cao nhất, LightGBM có Macro F1 cao nhất, còn Logistic Regression train nhanh nhất. Tuy nhiên, cả nhóm vẫn có Macro F1 thấp. Kết quả quan trọng của phân tích là phát hiện ảnh hưởng của lớp OnTime, không phải lựa chọn một mô hình có Accuracy nhỉnh hơn rồi coi bài toán nguyên nhân trễ đã được giải quyết.

Tổng train CPU 534,83 giây và toàn trình Spark 295,03 giây không có cùng phạm vi. Cộng ETL 66,94 giây với sáu dòng thời gian Spark cũ cho 345,39 giây, còn bao gồm các dòng chưa train thực đo. Sự không nhất

quán này không được giải quyết bằng cách chọn tổng nhỏ hơn. Nhóm không sử dụng chúng để công bố tăng tốc toàn trình.

B Giới hạn về thiết kế thực nghiệm

CPU và Spark sử dụng cách chia tập khác nhau, các tham số chưa được chứng minh tương đương và không có đủ log tài nguyên từng máy. Những yếu tố đó giới hạn diễn giải nhân quả về tốc độ. Số liệu hiện có trả lời mô hình chạy như thế nào trong thiết lập đã dùng, chưa trả lời riêng việc tăng số máy sẽ làm giảm bao nhiêu thời gian.

Nhãn nguyên nhân chi phối được tạo bằng quy tắc ưu tiên số phút lớn nhất, với Carrier làm mặc định trong một số trường hợp thiếu nguyên nhân. Quy tắc này đơn giản nhưng có thể đưa sự không chắc chắn của dữ liệu vào nhãn học máy. Thêm vào đó, các đặc trưng đầu vào chủ yếu là lịch bay và danh mục; chúng thiếu thông tin sự cố và thời tiết tại thời điểm bay.

Các bảng RAM không có phép đo đỉnh liên tục và không đại diện tổng bộ nhớ phân tán. Độ trễ CPU/Spark cũng chưa dùng cùng lô dữ liệu và cùng phạm vi thao tác. Một báo cáo đầy đủ phải nêu các giới hạn này ngay trong phần kết quả, thay vì trình bày tỷ lệ tốc độ như bằng chứng tuyệt đối cho tính ưu việt của Spark.

C Thứ tự cải thiện đề xuất

Bước tiếp theo nên tạo một split cố định theo định danh dòng, fit encoder trên train và lưu đầy đủ pipeline. Sau đó, Random Forest CPU và Spark cần cấu hình đối chứng rõ về số cây, độ sâu, đặc trưng và phạm vi đo. Mỗi lượt lưu phiên bản code, tham số, số mẫu, dự đoán và trạng thái, giúp kết quả có thể kiểm tra và tái lập.

Tiếp theo là nghiên cứu mất cân bằng lớp. Nhóm có thể thử trọng số lớp hoặc lấy mẫu lại trên tập train, giữ test nguyên trạng và theo dõi Macro F1 cùng recall từng nguyên nhân. Đây là đề xuất cần kiểm chứng, không phải cải tiến đã hoàn thành. Thêm nhiều đặc trưng cũng chỉ có ý nghĩa nếu chúng có sẵn trước thời điểm cần dự đoán và không làm rò rỉ nhãn.

Đối với Spark, cần chuyển đường đọc phù hợp sang hệ thống lưu trữ dùng chung khi triển khai trên nhiều máy, giảm tập trung dữ liệu ở Driver và đo tài nguyên từng Executor. Đối với Web_Air, cần benchmark tải và phân biệt xác suất model với mức tin cậy đã được hiệu chuẩn. Các bước này phục vụ chất lượng và khả năng triển khai, chưa được xem là một thuật toán mới.

D Đối chiếu kết quả với mục tiêu đề tài

Về xử lý dữ liệu lớn, đồ án có nguồn hơn bảy triệu dòng và đầu ra hơn sáu triệu dòng sạch. Về thuật toán, nhóm có Random Forest và các baseline thực tế trên CPU, cùng kết quả MLlib để đối chiếu trên Spark. Về ứng dụng, Web_Air nạp tài nguyên đã lưu và cung cấp một đường sử dụng cụ thể. Đây là các thành phần có thể kiểm tra, nhưng không đồng nghĩa mọi yêu cầu về chất lượng triển khai đã được giải quyết.

Điểm cần cải thiện của phép so sánh là sự đồng nhất về split, tham số và phạm vi đo. Điểm cần cải thiện của mô hình là khả năng phân biệt nguyên nhân trễ. Hai vấn đề nên được xử lý riêng: hoàn thiện đối chứng giúp kết luận đáng tin cậy hơn, còn cải thiện đặc trưng và cân bằng lớp có thể giúp tăng chất lượng. Không nên lấy việc chạy nhanh hơn làm bằng chứng rằng model nhận diện nguyên nhân tốt hơn.

Nếu phát triển một biến thể Random Forest, nhóm cần mô tả thay đổi cụ thể so với bản gốc, lý do lựa chọn và thực nghiệm với baseline. Một thay đổi về môi trường chạy hoặc giao diện web chưa phải tính mới thuật toán. Vì bộ thực nghiệm hiện tại chưa có biến thể đã kiểm chứng, báo cáo giữ kết luận về ứng dụng và triển khai, không tuyên bố đóng góp mới về mô hình học máy.

E Kết luận và lời cảm ơn

Nhóm đã hoàn thành một quy trình ứng dụng Random Forest trên dữ liệu lớn, có đối chiếu baseline về chất lượng và thời gian trên CPU cùng kết quả Spark thực đo. Phân tích cho thấy cần cải thiện nhận diện nguyên nhân trễ và kiểm soát thực nghiệm trước khi kết luận lợi ích phân tán. Đồ án không công bố tính mới thuật toán khi chưa có mô hình cải tiến và bằng chứng so sánh tương ứng.

Nhóm xin cảm ơn TS. Phan Hồ Viết Trường đã hướng dẫn và góp ý trong quá trình thực hiện đồ án. Nhóm cũng cảm ơn các thầy cô Khoa Công nghệ Thông tin và Trường Đại học Công Thương TP. Hồ Chí Minh đã hỗ trợ việc học tập và thực hiện đề tài.

TÀI LIỆU THAM KHẢO VÀ NGUỒN THỰC NGHIỆM

[1] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. Nguồn trực tuyến.

[2] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in Proc. 9th USENIX NSDI, 2012, pp. 15–28. Nguồn trực tuyến.

[3] X. Meng et al., "MLlib: Machine Learning in Apache Spark," Journal of Machine Learning Research (JMLR), vol. 17, no. 34, pp. 1–7, 2016. Nguồn trực tuyến.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," Commun. ACM, vol. 51, no. 1, pp. 107–113, 2008, doi: 10.1145/1327452.1327492. Nguồn trực tuyến.

[5] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in Proc. 19th ACM SOSP, 2003, pp. 29–43, doi: 10.1145/945445.945450. Nguồn trực tuyến.

[6] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD, 2016, pp. 785–794, doi: 10.1145/2939672.2939785. Nguồn trực tuyến.

[7] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 3146–3154. Nguồn trực tuyến.

[8] L. Prokhorenkova et al., "CatBoost: Unbiased Boosting with Categorical Features," in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018, pp. 6638–6648. Nguồn trực tuyến.

[9] Bureau of Transportation Statistics, "Reporting Carrier On-Time Performance (1987–present)." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[10] Federal Aviation Administration, "Air Traffic By The Numbers." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[11] Apache Software Foundation, "Apache Spark: Cluster Mode Overview." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[12] Apache Software Foundation, "Machine Learning Library (MLlib) Guide." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[13] Apache Software Foundation, "HDFS Architecture." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[14] Apache Software Foundation, "Apache Parquet: File Format." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[15] S. Ramírez, "FastAPI." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[16] Tailscale Inc., "What is Tailscale?." [Online]. Truy cập ngày 14/10/2026. Nguồn trực tuyến.

[17] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Inf. Process. Manage., vol. 45, no. 4, pp. 427–437, 2009, doi: 10.1016/j.ipm.2009.03.002. Nguồn trực tuyến.

[18] Y. Sun, A. K. C. Wong, and M. S. Kamel, "Classification of imbalanced data: A review," Int. J. Pattern Recognit. Artif. Intell., vol. 23, no. 4, pp. 687–719, 2009, doi: 10.1142/S0218001409007326. Nguồn trực tuyến.

[19] R. Hendrickx, M. Zoutendijk, M. Mitici, and J. Schäfer, "Considering airport planners' preferences and imbalanced datasets when predicting flight delays and cancellations," in Proc. IEEE/AIAA DASC, 2021, doi: 10.1109/DASC52595.2021.9594367. Nguồn trực tuyến.